

I

then go through the normal prioritization and planning process. The best way to avoid scope creep

is to have thorough, complete requirements in the issue when you initially open it. The issue templates should help guide you to include all relevant information.

The threshold for a "material amount of work" is to be determined by the analyst working on the issue.

Blocked issues

If an issue is blocked and it requires additional work to diagnose or troubleshoot (ex: a data issue is uncovered), a new issue should be opened, assigned a weight and priority, and linked to the original blocked issue. The new issue can be added to the current iteration without going through the formal planning process at the analyst's discretion, but this can impact our ability to complete all issues in the iteration.

The threshold for "additional work" is to be determined by the analyst working on the issue.

Experimentation analysis issues are naturally blocked by the experiment actually running (we have

to wait until we have sufficient data in order to perform the analysis). In order to enable a more

accurate measure of velocity, we will divide the work into 2 separate issues:

1. Experiment prep (dashboard creation and data validation)

1. Experiment analysis

At this time, PMs should continue to open a single issue and analysts will separate accordingly.

Undocumented requests

Please open an issue for all Product Data Insights requests. Requests made via comments in Google Drive are extremely difficult to track, and Slack history is gone after 90 days.

In addition, these requests are not planned or accounted for in team velocity. Your informal request might mean that we are unable to complete work we committed to for another stakeholder.

By keeping a formal record of requests (via issues), we are able to identify frequently asked questions (which could lead to building a self-service solution) and quickly replicate past work.

Capacity

The Product Data Insights team is tasked with supporting the entire Product organization, in addition to other product-related data needs across GitLab. As such, team capacity can be limited as we grow towards our target gearing ratio.

However, limited capacity should not stop GitLab team members from opening issues for Product Data Insights,

it simply means that lower-priority requests will have to wait until resources are available.

As the group grows, so will our ability to turn around requests in a shorter period of time.

Engaging the team

In addition to opening an issue, there are several other methods to engage with the team, both sync and async.

Office hours

In order to support more PMs across GitLab, the Product Data Insights team offers office hours. Office hours are held every other Wednesday, alternating between `<time datetime="16:00">4 pm UTC (11 am ET / 8 am PT)</time>` and `<time datetime="21:00">9 pm UTC (4 pm ET / 1 pm PT)</time>`. While our primary stakeholders are PMs across the organization, all GitLab team members are welcome to join.

The intent of office hours is to give PMs faster access to the team and get support for smaller tasks, brainstorming, and data self-service. More formal requests that answer more complex questions are captured in issues and go through a more rigorous, structured prioritization process.

How to sign up

The agenda

is first-come, first-served. Walk-ins/drop-ins are always welcome, but if possible, please add your name and topic (or question) before office hours begins. This allows the team time to review new agenda items ahead of time. If we are unable to cover a topic on the agenda, it will be pushed to the following meeting.

Stakeholders are welcome to leverage office hours to discuss and define new issues, which can help reduce async back-and-forth communication in the issue itself.

Example topics

Office hours are intended for smaller bodies of work, brainstorming, and assistance with data self-service. Here are some examples of topics for office hours:

`<details markdown="1"><summary>Example Topics</summary>`

`<details markdown="1"><summary>· Example Topic 1: Experiment Setup</summary>`

I am interested in launching an experiment to see if we can increase adoption of Secure.

- How would you go about setting the experiment up?
- Can you help me calculate the sample size?
- Can you help me interpret the results?

`</details>`

`<details markdown="1"><summary>· Example Topic 2: Approach to Analysis</summary>`

I am trying to do an analysis on the relationship between users with SSO enabled and invite acceptance rate.

- Which tables should I use? Can you help me understand this data source?
- What approach would you take?
- Would this metric answer the question?
- Can you help me understand this data source?

</details>

<details markdown="1"><summary>· Example Topic 3: Code Review</summary>

I wrote a query to calculate xMAU for namespaces that converted from a trial to a paid plan.

- Is this JOIN correct?
- Does this logic only include namespaces that had trials before converting?

Note: We are not able to accommodate all code reviews in the scope of office hours. Please limit this type of topic to specific aspects of a query, whether you are using the correct data source, etc.

</details>

<details markdown="1"><summary>· Example Topic 4: Dashboard Updates</summary>

I am looking to make some updates or enhancements to this existing dashboard.

- Can you help me incorporate a filter into this dashboard that would allow me to limit the charts to activity within 30 days of namespace creation?
- Can you update this funnel to include this additional event?

</details>

<details markdown="1"><summary>· Example Topic 5: Follow-Up Questions</summary>

In the last key meeting, you presented an analysis on early trial adoption.

- Can you walk me through your methodology?
- Can you help me understand the implications of the data/analysis?

</details>

<details markdown="1"><summary>· Example Topic 6: Scope and Define New Issue</summary>

I am going to open an issue for a new analysis.

- Can we discuss the overall scope and details?
- What kind of information should I include in the issue?

</details>

</details>

If the topic you bring is too broad to be addressed during office hours, we can discuss the work and will redirect you to open an issue.

FAQs

What is the difference between topics for office hours and formal data requests?

Office Hours is intended to help PMs with smaller tasks, provide a venue for brainstorming, and help folks looking to learn more about data self-service. The benefit is that the agenda is first-come, first-served, the prioritization process is bypassed, and the wait time is minimal.

Formal data requests and larger bodies of work are captured in issues in the Product Data Insights project.

They can help answer more complex questions, but go through more robust intake and planning processes. As such, there is a longer turn-around time given team size and capacity.

What if I don't know if my topic is best suited for office hours or whether I need to open an issue?

Feel free to ask your analyst partner (if applicable) or in data.
When in doubt, come to office hours and the team can discuss there.

Slack

Channels

- 1. data - For any type of data question, including those related to product and/or the Product Data Insights team
- 1. data-tableau - For any questions related to Tableau
- 1. gproductdatainsightsdaily - For the Product Data Insights team's asynchronous daily stand-up, powered by Geekbot

Aliases

- 1. @product-analysts - Notifies the entire Product Data Insights team
- 1. @randdanalyticstriage - Notifies the entire Product Data Insights team and the Data team's R&D sub-team, per the Enterprise Data Triage Program
- 1. @functional-data-analysts - Notifies the entire Product Data Insights team and other functional analysts across the GitLab Data Program

GitLab groups

- 1. @gitlab-data/product-analysts - Notifies the entire Product Data Insights team

YouTube playlist

1. Product Data Insights - Recordings from office hours, analysis/read-outs, etc

Other helpful resources & links

1. Data for Product Managers
1. Data Catalog/Data Guide Series (internal handbook)
1. Data Team Handbook

SECTION: product/groups/product-analysis/crash-course.md

Objectives for this page

This page is intended to provide a crash course style overview of the most important Product Analytics related resources for each product Stage.

As a Product Analyst or other curious GitLab team member, it can be helpful to have a quick and easy reference for each product Stage to quickly understand high-level functionality, key objectives or a distilled product roadmap, and key data resources currently used under a specific Stage or Group within GitLab before jumping in to an analysis.

If this page serves it's purpose, Product Analysts should be able to visit this handbook page before working with a Stage or Group to obtain helpful contextual information without needing to do a scavenger hunt search across dozens of handbook pages to find relevant information.

Helpful places to start

- Internal handbook performance indicators (PIs) - This internal handbook page is a great place to understand the PIs for each Stage / Group to understand what results each team is trying to achieve.
- Product categories handbook page contains a huge amount of helpful information and defines sections, stages, groups, and categories.
- Need to quickly learn how a certain Stage/Group feature works? GitLab Docs
- Metrics dictionary contains metrics and events collected in Service Ping and Snowplow. For more information visit the dedicated Analytics Instrumentation Guide.
- Features by Group handbook page contains a tabular breakdown of features included in each Stage Group by Core, Premium or Ultimate categories.
- Definitive guides to data subject areas managed by the Data team.
- Centralized Metrics: Key reporting dashboard for GitLab product usage metrics.
- The Technical Marketing handbook page contains links to product demos, webinars and release

videos.

- GitLab's Fiscal Year runs from February 1 to January 31. Here is a link to GitLab's Fiscal Quarter to Month map.

Learn about analytics relevant product stage topics grouped by Section

Why are these groupings called 'Stages'? At GitLab, we build a product that supports every stage of the DevOps Lifecycle.

Dev Section

Manage Stage

This Stage helps teams manage their software delivery lifecycle with metrics and value stream insight in order to streamline and increase their delivery velocity. Insights, authentication, audits, compliance, user management, group & sub-group management, and project management.

<details markdown="1"><summary>Click to expand</summary>

Top dashboards referenced by this team

- Centralized Metrics: Key reporting dashboard for GitLab product usage metrics.

Helpful video resources

Plan:Optimize YouTube Channel

Team meetings, AMAs, etc. for the Plan:Optimize Group

Manage:Access YouTube Channel

Team meetings, AMAs, etc. for the Manage:Access Group

Manage:Organization YouTube Channel

Team meetings, AMAs, etc. for the Manage:Organization Group

Product direction

Problems to Solve from Product Direction Page

The Problems to Solve section of Manage's Product Direction Page is the most condensed documentation to understand Manage's big picture focus quickly.

Performance indicators

Metrics from Product Direction Page

This link breaks down the performance indicators for each Manage Group clearly and links out to

the Internal Handbook PI page.

Key handbook pages

Manage Direction Page

Main Manage Stage Page

Slack channels

smanage

Team members

Handbook page to find Manage team members to collaborate with

</details>

Plan Stage

Enable teams to effectively plan and execute work in a single application. This stage enables portfolio planning and management through epics, groups (programs), milestones, etc. to organize and track progress

<details markdown="1"><summary>Click to expand</summary>

Top dashboards referenced by this team

- Centralized Metrics: Key reporting dashboard for GitLab product usage metrics.
- Plan stage .com events Analyses dealing with Snowplow events tables
- Fulfillment:Platform PIs Dashboard
- Growth Section Analytics Hub
- Code Suggestions
- Verify Performance Indicator Hub

Helpful video resources

Plan Stage YouTube playlist

What the Plan Stage team is working on

Link to 1 Year Plan

A great handbook page to skim to understand the roadmap and focus for Plan

Performance indicators

Link to performance indicators

Resource to understand the Performance Indicators for the Dev Section - Page can be searched by Stage and Group

Key handbook pages

Main Plan page

Plan direction page

Slack channels

splan

Team members

Handbook page to find Plan team members to collaborate with

</details>

Create Stage

Create provides tools that support the beginning of the DevOps Life Cycle. Source Code management, Code Review, Wiki, Web IDE, etc.

Start on the GitLab website page dedicated to Create to get an understanding of features included in this Stage.

<details markdown="1"><summary>Click to expand</summary>

Top dashboards referenced by this team

- Centralized Metrics: Key reporting dashboard for GitLab product usage metrics.

- Performance indicators internal handbook page The Create : GitLab Group primarily uses the Performance Indicators Internal Handbook Page to guide decisions

Helpful video resources

Must be logged into GitLab Unfiltered account

Create Stage YouTube playlist

Create Stage UX YouTube playlist

Product roadmap link

Link to product direction / vision

Resource to understand the goals for this team

Performance indicators

Link to performance indicators

Resource to understand the Performance Indicators for the Dev Section - Page can be searched by Stage and Group

Key handbook pages

Primary Create Stage handbook page

Contains helpful information about how Create operates and current team members

Slack channels

screate

screatepm

Team members

Handbook page to find Create team members to collaborate with

</details>

CI Section

Verify Stage

Verify provides teams with tools that help facilitate continuous integration. This includes but is not limited to pipelines and runners.

<details markdown="1"><summary>Click to expand</summary>

Top dashboards referenced by this team

- Centralized Metrics: Key reporting dashboard for GitLab product usage metrics.
- Verify Performance Indicator Dashboard
- Error Budget Dashboard

Important data documentation

There are a few models that are exclusive to the Verify stage that you can reference as a SSOT:

- `wkgitlabdotcommonthlycicomputeminutes` is a model you can use to identify the number of runners, pipelines, and CI Build Minutes used on a per project, per namespace level.
Helpful video resources

Tanuki Tech: Verify and Secure

This video showcases the product functionality of Verify and Secure and how to talk about those functionalities from a sales perspective.

Verify Team Overview

Product roadmap link

Section Direction: Verify Stage

Resource to understand the long-term goals for the Verify team

Performance indicators

Ops Section PI

An internal handbook page that lists all performance indicators under the Operations section

Key handbook pages

Verify Stage Product Page

A central hub for all pages related to the Verify stage

Slack channels

`sverify`

Overall channel for Verify

`gpipeline-execution`

Slack channel for the Verify:Pipeline Execution product category.

`gpipeline-authoring`

Slack channel for the Verify:Pipeline Authoring product category.

`grunner`

Slack channel for the Verify:Runner product category.

`gpipeline-security`

Slack channel for the Verify:Pipeline Security product category.

Team members

Handbook page to find Verify team members to collaborate with

</details>

Package Stage

The Package team works on the package and container registries, as well as the Dependency Proxy.

<details markdown="1"><summary>Click to expand</summary>

Top dashboards referenced by this team

- Package Performance Indicator Hub

Primarily time series analyses for Package features on GitLab.com

- Centralized Metrics: Key reporting dashboard for GitLab product usage metrics.

Important data documentation

- wkgitlabdotcompackageevents is a model you can use for all Snowplow events related to Package. This includes but is not limited to, package actions for all available Package types on GitLab.com

Helpful video resources

User Interviews YouTube Channel

General and feature specific user interviews for the Package team

Demos and Speedruns

Package channel featuring demos, speed-runs and other content

Product roadmap link

Link to product roadmap

Resource to understand the long-term goals for this team

Key documentation

Main Package Team Handbook Page

It can be helpful to search for specific topics on the Package team's main page

GitLab Docs Package Page

GitLab Docs are awesome!

Slack channels

spackage

Team members

Handbook page to find Package team members to collaborate with

List of Package team members and their stable counterparts to contact if needed

</details>

CD Section

Deploy Stage

The Deploy team covers the deployment and release functionality of GitLab

<details markdown="1"><summary>Click to expand</summary>

Top dashboards referenced by this team

- Centralized Metrics: Key reporting dashboard for GitLab product usage metrics.
- Deploy Performance Indicator Dashboard: Primarily time series analyses for Deploy features on GitLab.com

Product roadmap link

Product Direction - Delivery

Resource to understand the long-term goals for this team

Performance indicators

Link to performance indicators

Resource to understand the Performance Indicators for the Ops Section - Page can be searched by Stage and Group

Key handbook pages

Deploy Primary Handbook Page

Slack channels

cd-section

genvironments

sdeploy

Team members

Handbook page to find Create team members to collaborate with

</details>

Sec Section

Secure Stage

Static Application Security Testing (SAST), Dynamic Application Security Testing (DAST), Container Scanning, and Dependency Scanning

<details markdown="1"><summary>Click to expand</summary>

Top dashboards referenced by this team

- Dynamic Analysis Metrics Secure:Dynamic Analysis Group

Some seemingly duplicate charts in Dynamic Analysis Metrics are used by the team to compare totals from different data sets.

- Threat Management Metrics Govern:Threat Insights Group

- Centralized Metrics: Key reporting dashboard for GitLab product usage metrics.

Helpful video resources

DevSecOps Overview A great place to start for a high-level overview relating to Secure functionality.

Product direction link

Secure product direction page - 1 year plan

Performance indicators

Performance indicators linked in Secure handbook page

Key handbook pages

Secure stage primary handbook page

Slack channels

ssecure

Team members

Product categories page section to find Secure team members to collaborate with

Secure Handbook page section to find Secure engineering team members to collaborate with

</details>

Software Supply Chain Security Stage

Manage security vulnerabilities, policies, and compliance across your organization.

<details markdown="1"><summary>Click to expand</summary>

Top dashboards referenced by this team

- Centralized Metrics: Key reporting dashboard for GitLab product usage metrics.
- PD: Sec: Software Supply Chain Security Metrics

Helpful video resources

Software Supply Chain Security Stage YouTube Channel

Software Supply Chain Security UX YouTube Channel

Product direction link

Software Supply Chain Security product direction page - 1 Year Plan

Performance indicators

Internal handbook performance indicators for the Secure section

Key handbook pages

Software Supply Chain Security stage primary handbook page

Slack channels

ssoftware-supply-chain-security

Team members

Handbook page to find Software Supply Chain Security team members to collaborate with

Software Supply Chain Security Handbook page section to find Software Supply Chain Security engineering team members to collaborate with

</details>

Core Platforms Section

Systems Stage (Distribution, Gitaly, Geo Groups)

Supports enterprise-grade operational experience of GitLab products from streamlined deployment and maintenance, disaster recovery, secure search and discoverability, to high availability, scalability, and performance. The Systems Stage includes Distribution, Gitaly and Geo Groups.

<details markdown="1"><summary>Click to expand</summary>

Top dashboards referenced by this team

- All Tableau Dashboards are organized in Collections, specifically Core & SaaS Platforms General Collection and Core & SaaS Platforms SAFE Collection

Important data documentation

There are no Enablement metrics currently collected in our Postgres Replica data for GitLab.com

The following tables can be used for Service Ping metric reporting.

- commonmart.martpinginstancemetricalltime - Use for all time timeframe metrics
- commonmart.martpinginstancemetric7day - Use for 7 day timeframe metrics
- commonmart.martpinginstancemetric28day - Use for 28 day timeframe metrics
- commonmart.martpinginstancemetricmonthly - Use for all, 7, 28 timeframe metrics pre filtered to the last ping of the month (does not include none or null timeframe metrics)
- workspaceproduct.wkfctpinginstancemetricnone - Use for metrics with a none timeframe
- workspaceproduct.wkfctpinginstancemetricnull - Use for metrics with a null timeframe

Use the metrics dictionary to determine the timeframe value for any service ping metric.

Helpful video resources

Enablement::Systems is a relatively new Stage. PDI will add helpful video overviews as they become available.

Product roadmap link

Enablement Product Direction
Includes Stage and Group level details

Performance indicators

Engineering PI handbook page

Key handbook pages

Overall Data Access Section Handbook page for Engineering

Slack channels

senablement

gdistribution

ggeo

Team members

Handbook page to find Systems team members to collaborate with

OR

All team members section of engineering page

</details>

Growth Section

Although the Growth Section falls within the Marketing & Strategy division, the Growth Section has dedicated support from Product Data Insights on topics within our ownership areas.

Growth Stage (Acquisition, Activation Groups)

Growth owns the free and trial registration and new user onboarding experiences.

<details markdown="1"><summary>Click to expand</summary>

Top dashboards referenced by this team

- All Tableau Dashboards are organized in Collections, specifically Growth General Collection and Growth SAFE Collection
- Growth Section Product Data Insights Hub in Tableau
- Filterable Growth Experiment Analysis Dashboard
- Growth Experiment Event Validation
- Snowplow Event Exploration L30D - commonly sent to engineers for event validation.

Important data documentation

- workspaceproduct.wkrptnamespaceonboarding This model contains all of the most commonly analyzed Growth use cases including namespace and namespace creator attributes and SaaS product behavior. This model can be contributed to by PDI, DEX and Marketing Analytics teams.
- Because Growth topics are broad and cross-functional, there are many data models that are commonly used to answer Growth product analytics questions. More comprehensive documentation can be found in Product Data Insights Data Models Cheat Sheet.

Helpful video resources

Growth YouTube playlist

Product roadmap link

Growth Direction handbook page

Performance indicators

- SaaS Team Activation

- Valuable Signup

- Both of the above metrics are monitored in Growth Section Product Data Insights Hub in Tableau

Key handbook pages

Overall Growth Section Handbook page for Engineering

Growth Direction Page

Slack channels

sgrowth

Team members

All team members section of engineering page

</details>

SECTION: product/groups/product-analysis/dashboards-analysis-insights.md

Dashboards

Here is a list of Tableau dashboards owned by the Product Data Insights team. Note:

Viewing dashboards in the SAFE project require additional steps to obtain SAFE Dashboard access (if you do not already have SAFE access in Tableau)

This list includes dashboards in draft. Dashboards carrying the Draft tag should not be used for reporting.

Dashboard Name	Description	Date	Location	SAFE Access Required?	
----------------	-------------	------	----------	-----------------------	--

---	---	---	---	---	
-----	-----	-----	-----	-----	--

Product Data Insights Collection	This is a Tableau Collection containing analytics resources that are broadly useful to the Product Org.	Ongoing	Product/General &		
----------------------------------	---	---------	-------------------	--	--

Product/SAFE | Y & N |

| PDI: AI Collection | This is a Tableau Collection containing PDI-owned resources related to AI feature adoption and usage. It contains both general and SAFE workbooks. | Ongoing | Product/General & Product/SAFE | Y & N |

| PD: Cloud Licensing Adoption Dashboard | This dashboard provides insights on adoption of Cloud Licensing | FY24 Q1 | Ad-hoc/Product/SAFE | Y |

| PD: Centralized Product Usage Metrics | Monthly reporting metrics and trends for product performance indicators | FY24 Q4 | Ad-hoc/Product/General | N |

| PD: Firmographic Product Metric Usage | Segment metric usage by company size, industry, and seat count. | FY24 Q4 | Development/Product/SAFE | Y |

| PD: Code Suggestions | Code Suggestions usage (ex: User Count), quality (ex: Acceptance Rate), and performance (ex: Load Time) metrics over time by delivery, language, etc. | FY24 Q4 | Ad-hoc/Product/General | N |

| PDI: Growth Section Collection | Contains workbooks and views that are relevant to the Growth Section | Ongoing | Product/General & Product/SAFE | Y & N |

| PDI: Core & SaaS Platforms Sections Collection | Contains workbooks and views that are relevant to Platforms Sections | Ongoing | Product/General & Product/SAFE | Y & N |

If you have any questions about using and/or interpreting the insights from these dashboards, please reach out to the dashboard owner or tag @product-analysts in the data channel on Slack.

Analysis & Insights

| Analysis | Description | Date |

| --- | --- | --- |

| Groups per Organization Metric Exploration | Establishes usage benchmarks (days used per month and consecutive months used) for group event usage. | FY24 Q1 |

| Release AMAU Decline Investigation | What caused the decline of Release AMAU from November 2022? | FY24 Q1 |

| Average Environments for Namespaces and Projects | This analysis covers what the average number of environments and environment variables projects and namespaces on GitLab.com use | FY24 Q1 |

| Payments declined due to lack of 3D Secure Authentication | This analysis gives us insights on how many payments are declined because we lack 3D Secure functionality | FY24 Q1 |

| Understanding user accounts with invalid zip/city combination | This analysis gives us insights on the impact of how many customers can not purchase a subscription if they have an invalid zip/city combination | FY24 Q1 |

| SaaS Team Activation Metric 2022 | Covers the development of a Growth and Marketing metric used to identify team actions performed in the 1st 14 days that are predictive of paid conversion and retention. | FY23 Q4 |

| Secrets Management Usage Spike | What caused a spike in Secrets Management Usage in August 2022? | FY23 Q3 |

| Investigation How Remote:Include Timeouts on GitLab.com | This analysis investigates why there was a large spike in HTTP 503 errors when running pipelines | FY23 Q4 |

| Verify Benchmark Analysis | This analysis was done to get an understanding of what typical test environments look like for Verify users | FY23 Q4 |

| Delayed Value Analysis | Studies key feature usage among Trial-to-Paid conversions and presents how trial feature adoption expedites paid feature adoption. | FY23 Q4 |

| CI Runner Usage for Accounts with 1000+ Seats | Study of the CI Runner Usage of large accounts with more than 1000+ | FY23 Q4 |

| Analysis of User Caps feature usage and account growth | This analysis gives us insights on correlation between 'User Cap' feature usage and growth metrics| FY23 Q4 |

| Analysis on behavior of Change Linked Namespace flow | This analysis gives us insights on how many SaaS subscriptions change their assigned Namespace| FY23 Q4 |

| Characterize CI -> SAST Funnel among Free Projects |Analysis to understand how SAST usage churns over time, and how much of this churn is due to dormant projects (that aren't using CI) vs. active projects that disable SAST. | FY23 Q3 |

| Relationship between Sec Section Features & Ultimate Customer's Purchase Decisions | Dissection of post purchase survey data to identify how important Sec features were in motivating Ultimate plan purchases. | FY23 Q3 |

| Free vs. Paid. vs OSS/EDU Subscriptions: Engagement Comparison | Comparison of similar metrics (SpO, user-stage engagement, Service Ping Opt-In rate, version adoption rate, etc) based on subscription type (Paid, Free, OSS/EDU) | FY23 Q3 |

| Verify Usage Patterns for Legitimate users & Malicious Users | Study of initial concurrent pipeline & job usage among free accounts to determine benchmarks between legitimate and malicious users | FY23 Q3 |

| Valuable Signup Metric 2022 | Covers the development of a Growth and Marketing metric used to narrow down to a subset of the GitLab.com user population with a higher likelihood of converting to paid. | FY23 Q3 |

| Understand User Adoption of Truncating Container Registry Image Name | A deep dive into the usage of truncating registry image names to determine rollout to all users | FY23 Q3 |

| SaaS & Self-Managed Purchase Funnel Analysis Overview | Deep dive analysis with multiple insights for SaaS & Self-Managed purchase funnel| FY23 Q2 |

| FY23-Q2 xMAU Estimation Exploration| Exploration into viability of different Self-Managed estimation methodologies using data models available in July 2022 | FY23 Q2 |

| Release Stage Retention: SaaS vs Self-Managed | What is causing the large discrepancy between Release retention on SaaS vs Release retention on Self-Managed? | FY23 Q2 |

| Projects and Namespaces that use GitLab.com Terraform State | A deep dive into the percentage of projects and namespaces use the GitLab.com Terraform state. | FY23 Q2 |

| User Impact of Turning Off Certificate-Based Clusters | How many users (internal and external) will be affected by turning off certificate-based clusters? | FY23 Q1 |

| SaaS Trial Firmographic Analysis | | FY22 Q4 |

| FY22-Q3 xMAU Estimation Exploration | Exploration into viability of different Self-Managed estimation methodologies using data models available in August 2021 | FY22 Q3 |

| Paid SpO Trends among Retained and Churned Namespaces | What stage adoption trends are present when looking at churning namespaces vs.retained namespaces? | FY22 Q3 |

| SSO Login Deep Dive Analysis | Takes a deep look into registrations that are created through SSO and what their subsequent product usage looks like | FY22 Q2 |

| Secure Free-to-Paid Funnel and Feature Adoption Analysis | Which Secure features are the most adopted during trials and which features have the highest correlation with Ultimate plan paid conversion? | FY22 Q2 |

| Stage Adoption Patterns: SCM <> Code Review <> CI | How do organizations go from using SCM to Verify, and is Code Review used between or after these stages? | FY22 Q1 |

SECTION: product/groups/product-analysis/data-model-cheat-sheet.md

Objectives for this page

This handbook page is intended to provide a high-level overview of the most common data models used by the Product Data Insights team as well as some known nuances and/or caveats about those data models that are helpful to be aware of.

To collaborate on the content in this page, please either submit an MR (preferred) or start a discussion in this Epic.

Helpful places to start

- DBT Docs - This resource contains comprehensive documentation on all available dbt models. When in doubt, search DBT!
- Data guides to data subject areas managed by the Data team.
- Documentation on data pipelines for the technically curious analyst. This page goes into each data source and extraction details.
- Table of data sources and refresh schedules to understand standard load times for each data source.
- Enterprise Data Catalog to understand enterprise analytics subject areas that are broadly useful to the GitLab organization.

Data Model Categories

These categories are grouped by data source and subject area.

Service Ping

Service Ping is GitLab's mechanism to collect data by generating a JSON payload of usage data every week to be sent to GitLab. It provides aggregated data to our Product, Customer Success, Support, and Sales teams to understand how GitLab is used. Service Ping is our only data source for understanding Self-Managed product behavior. Service Ping methodology allows us to protect our Self-Managed users' privacy by aggregating metrics at the installation level.

FAQs

> Is it possible to report at the namespace or user level using Service Ping data?

- Nope! As part of our Commitment to Individual User Privacy, GitLab only collects usage metrics aggregated at the installation level.

> What is the difference between an instance and an installation?

- An installation is the unique combination of instanceid and hostid. Read more here. We do Self-Managed analysis and reporting at the installation level.

Documentation

<details markdown="1"><summary>Click to expand</summary>

- Service Ping Overview
- Service Ping metrics dictionary
- Data Guide to Self-Managed Analysis
- Data Guide to xMAU Analysis

</details>

Commonly Used Data Models

<details markdown="1"><summary>Click to expand</summary>

Schema	Table Name	Data Grain	Description	Notes
---	---	---	---	---
commonmart	martpinginstance	dimpinginstanceid	Ping-level data with information with additional attributes for installation, subscription, account, and product information.	No metrics are included in this data.
commonmart	martpinginstancemetric	dimpinginstanceid, metricspath	Ping- and metric-level data with additional attributes for installation, subscription, account, and product information.	This is a UNION of other tables that are filtered by a certain timeframe: martpinginstancemetric28day martpinginstancemetric7day martpinginstancemetricalltime
common	fcptpinginstancemetricnonenull	dimpinginstanceid, metricspath	Ping- and metric-level data about metrics with none and null timeframes.	
commonmartproduct	rptpinglatestsubscriptionsmonthly	pingcreateddatemonth, latestsubscriptionid, diminstallationid	Active Self-Managed subscriptions by month, including seat count. If a subscription sends Service Ping, then installation-level data is provided.	This includes seat count and can be used to calculate Service Ping opt-in rate
commonmartproduct	rptpingmetrictotalswestimatesmonthly	pingcreateddatemonth, metricspath, pingedition, estimationgrain, pingeditionproducttier, pingdeliverytype	This model is used for xMAU/PI reporting and is the source for Service Ping data in the [tdxmau] snippet.	

</details>

Good to Know

<details markdown="1"><summary>Click to expand</summary>

- Categories of data collected: Subscription, Operational, Optional
 - Operational metrics
 - Optional metrics
- Installations are randomly assigned a day of week to generate service pings, but that assignment is persistent over time. For example, if an installation is assigned Tuesdays to generate pings, it will always generate pings on Tuesdays. We generate and load service ping on different days to distribute the payload load evenly over the entire week.
- The milestone field of the metrics dictionary can also be used to identify the version when a metric was instrumented, but there a couple of limitations. First, many metrics are just

labeled < 13.9, so there is a lack of more detail for older metrics. Second, metrics can be introduced on different versions for CE and EE, so milestone could be incorrect for one edition/distribution. For these reasons, we recommend using `commonmartproduct.rptpingmetricfirstlastversions` if you are looking to find out when a metric was instrumented.

</details>

GitLab.com

GitLab.com (SaaS) is a single installation reporting a single ping within our Service Ping framework. In order to access more granular data by product tier, plan type, namespace, or user, we utilize the GitLab.com Postgres database. This data source replicates any service ping events that create a backend table.

FAQs

> Why don't all events for all Stages and Groups show up in our GitLab.com data?

- This is due to limitations in replicating Service Ping counters using the gitlab.com db Postgres replica

> Is it possible to associate user level behavior in our GitLab.com data to our Snowplow events?

- No. Our snowplow user identifiers are anonymized, while our GitLab.com user identifiers are not. However, it is possible to join Snowplow and GitLab.com data at the namespace (group/project) level.

> I've heard there are some reliability issues with our GitLab.com data. How can I stay up to date on outages or known problems?

- This Issue documents all known problems with the GitLab.com replica.

Documentation

<details markdown="1"><summary>Click to expand</summary>

- Data Guide for Product Managers documentation on GitLab.com postgres replica data

- DB docs document which service ping metrics are replicated in a database. Click in to the .yaml files for each table to access table specific descriptions.

- DBT documentation on the prepevent model contains compiled SQL logic to better understand any filtering applied to events.

- Data Guide to xMAU Analysis

- Schema file containing SQL logic for the creation of each postgres table available in production.

</details>

Commonly Used Data Models

<details markdown="1"><summary>Click to expand</summary>

Schema	Table Name	Data Grain	Description	Notes
---	---	---	---	---
commonmart	marteventuserdaily	eventdate, eventname, dimuserid, dimultimateparentnamespaceid	Daily user-, namespace-, and event-level data, including attributes about the namespace and plan	
commonmart	marteventnamespacedaily	eventdate, eventname, dimultimateparentnamespaceid	Daily namespace- and event-level data, including attributes about the namespace and plan	
commonmartproduct	rpteventxmaumetricmonthly	eventcalendarmonth, usergroup, sectionname, stagename, groupname	Monthly user group- and xMAU metric-level data This is the model used in reporting paid SaaS xMAU and is used in the [tdxmau] snippet	
commonmartproduct	rpteventplanmonthly	eventcalendarmonth, planid, eventmonth, eventname	Monthly plan- and event-level data	

</details>

Good to Know

<details markdown="1"><summary>Click to expand</summary>

- GitLab.com data sources are not exhaustive of all of the actions users can take within GitLab's SaaS offering.

</details>

Snowplow

Snowplow is an open source event tracking tool that is used at GitLab to track GitLab.com front-end events like page views, CTA clicks, link clicks, etc. This data source does not collect identifiable user data to protect our user's privacy. Our Snowplow data source is how we implement and track experiments at GitLab.

FAQs

> Why doesn't the metric that my team implemented show up in the metrics dictionary?

- In order to show up in the metrics dictionary, every event needs a .yaml file. This will not happen automatically and should be created by the engineer that implements snowplow tracking.

> Why is the value for gscnamespaceid null for some proportion of snowplow events?

- Engineers need to enable tracking for gscnamespaceid when implementing new events. If tracking for gscnamespaceid is already enabled and nulls are still occurring, the events may be triggered in a location within GitLab.com that is not specific to any one namespace like the Todos page.

> What is the correct logic to identify events triggered in production environments?

- Apply the following logic WHERE appid IN ('gitlab','gitlabcustomers')

> How should I interpret the eventcategory value in structured snowplow event payloads?

- The eventcategory value will be automatically populated according to this codified logic unless the engineer instrumenting the event overrides this logic, which is often the case for backend events. A great place to search for the meaning of these values is by key word searching in this EE controllers repository. Controllers outside of EE are also searchable and located here. Otherwise, you can reach out to the engineering slack channel for the team who instrumented the event of interest and ask for validation on the correct interpretation of your eventcategory value there.

Documentation

<details markdown="1"><summary>Click to expand</summary>

- Guide to Snowplow for Product Managers

- Technical Snowplow overview

- Snowplow docs on standard fields

- PDI: Snowplow New Models Onboarding

</details>

Commonly Used Data Models

<details markdown="1"><summary>Click to expand</summary>

| Schema | Table Name | Data Grain | Description | Notes |

| --- | --- | --- | --- | --- |

| commonmart | martbehaviorstructuredevent | behaviorstructuredeventpk | Enriched Snowplow table for the analysis of structured events. | Depending on analysis use case, it could be helpful to filter by behaviordate for queries to run within reasonable timeframes. |

| common | fctbehaviorstructuredeventwithoutassignment | behaviorstructuredeventpk | Derived fact table containing data for Snowplow structured events excluding assignment events. Assignment events are events that signifies a user was enrolled into an Experiment. |

fctbehaviorstructuredeventwithoutassignment190 and

fctbehaviorstructuredeventwithoutassignment400 are also available. |

| common | fctbehaviorstructuredeventexperiment | behaviorstructuredeventpk | Derived fact table for structured events related to experiments. | |

| common | fctbehaviorwebsitepageview | fctbehaviorwebsitepageviewsk | Fact table containing quantitative data for Page views. Page views are a subset of Snowplow events and are fired by the Javascript tracker. | |

| common | fctbehaviorunstructuredevent | fctbehaviorunstructuredsk | Derived fact table for unstructured events. | |

| common | dimbehaviorevent | dimbehavioreventsk | Dimensional model containing distinct events

types from Snowplow. | |

</details>

Good to Know

<details markdown="1"><summary>Click to expand</summary>

- If you are wondering if Snowplow events are implemented in a certain area of the product, the Snowplow Inspector is a good complimentary resource to the Metrics Dictionary which is not exhaustive. The Snowplow Inspector will not show server side events.

- We do not use snowplow on our self-managed instances, only on GitLab.com

- If developers or PMs are wondering about standard implementation, the event schema is documented.

</details>

Namespaces, users, & memberships

This category of data models includes GitLab.com (SaaS) namespaces (which include both projects and groups), their firmographic attributes, and individual members.

FAQs

> What is a namespace?

- Starting with the basics! GitLab has two categories of namespaces; groups and projects. In general, a namespace provides one place to organize your related projects. Read more here. Namespaces exist within GitLab SaaS and Self-Managed products, but to protect the privacy of our Self-Managed users, we only collect identifiable namespace data for SaaS.

> What types of namespaces do we normally analyze?

- We normally perform analyses at the Ultimate parent namespace level.

> Do we have access to membership history data?

- No. Membership history at GitLab is not recorded in any data models.

Documentation

<details markdown="1"><summary>Click to expand</summary>

- Data Guide to Namespace Analysis contains comprehensive documentation on namespace analytics and example SQL code.

- This knowledge base page covers an overview of namespaces, members and groups.

- Member-specific knowledge base page explaining direct and indirect memberships as well as shared group memberships.

</details>

Commonly Used Data Models

<details markdown="1"><summary>Click to expand</summary>

Schema	Table Name	Data Grain	Description	Notes
---	---	---	---	---
common	dimnamespace	dimnamespaceid	Dimension table that contains all GitLab.com namespaces and namespace attributes including plan.	
common	dimnamespacehist	namespacesnapshotid, dimnamespaceid, validfrom, validto	Historical snapshot of common.dimnamespace model.	
common	dimuser	dimuserid	Dimension table that contains all GitLab.com Users.	
common	dimuserhist	dimusersnapshothistid, dimuserid, validfrom, validto	Historical snapshot of common.dimuser model.	
legacy	gitlabdotcommemberships	membershipsourcedid, userid	This model unions together all of the other models that represent a user having (full or partial) access to a namespace, AKA "membership". Includes both direct and indirect membership types.	
legacy	gitlabdotcommembers	memberid, userid	Base model for GitLab.com members. Only includes direct membership links. Used for invite related fields.	

</details>

Good to Know

<details markdown="1"><summary>Click to expand</summary>

- membercount fields found in any common models are not accurate and should not be used. Use legacy.gitlabdotcommemberships for any analyses intended to measure members per namespace. Here is the Issue representing work to correct these accuracy problems.

</details>

Duo

GitLab Duo is a suite of AI-powered features including Code Suggestions, Chat, and other capabilities. Data about Duo usage comes from multiple sources including AI Gateway events, Snowplow tracking, and Service Ping metrics, with the AI Gateway being the source of truth for usage metrics across all deployment types starting August 2024. For the most comprehensive documentation see Data Guide to Duo Analysis.

FAQs

> When can we report total deduped users across all Duo features?

- Complete deduped totals across all features and deployment types are only available starting August 3rd, 2024. Historical data availability varies by feature and deployment type.

> How is Duo usage attributed to customers?

- Usage is attributed based on how an event happened (which namespace/installation enabled access), not where the event occurred. A single event can be enabled by multiple customers.

Documentation

<details markdown="1"><summary>Click to expand</summary>

- Data Guide to Duo Analysis

</details>

Commonly Used Data Models

<details markdown="1"><summary>Click to expand</summary>

Schema	Table Name	Data Grain	Description	Notes
---	---	---	---	---
workspaceproduct	wkmartbehaviorstructuredeventaigatewayflattened	Event per namespace/installation	AI Gateway events with customer attribution	Flattened model - use DISTINCT counts
workspaceproduct	wkrptaigatewayeventsflattenedwithfeatures	Event per namespace/installation	AI Gateway events with customer attribution joined to the feature associated with each request.	Flattened model - use DISTINCT counts
commonmartproduct	rptbehaviorcodesuggestionoutcome	Suggestion	Code Suggestions metrics from IDE extensions	Quality metrics like acceptance rate
restrictedsafeworkspaceproduct	rptduolicenseutilizationmonthly	Subscription/month/add-on	License utilization metrics	Excludes current month
workspacecustomersuccess	wklicensebillableusers	Installation	Self-managed seat assignments	Available from v17.5+

</details>

Good to Know

<details markdown="1"><summary>Click to expand</summary>

- AI Gateway events in wkmartbehaviorstructuredeventaigatewayflattened and wkrptaigatewayeventsflattenedwithfeatures are the SSOT for usage metrics starting August 2024 and cannot be blocked by users

- Customer attribution relies on different identifiers:

- GitLab.com: featureenabledbynamespaceids
- Self-Managed & Dedicated: instanceid + hostname

- For improvements in progress on AI Gateway reporting, follow: <https://gitlab.com/gitlab-org/gitlab/-/issues/502457>

</details>

SECTION: product/groups/product-analysis/dbt-cheat-sheet.md

Objective of this page

This page is intended to be a cheat sheet or quick start guide to dbt for functional analysts. There is a wealth of documentation available in the Data Team dbt Guide, but it can be hard to sift through for folks who are newer to analytics engineering (ex: functional analysts).

This page aggregates existing resources in order to guide you through the basic steps required to create and test an MR to update a model in dbt.

dbt change workflow

Any development should follow the documented dbt Change Workflow. Please review that handbook page for details on how to approach making changes in dbt (planning, testing, etc).

Local setup

First things first, you need to get set up with dbt on your local machine. Please read the Configuration section of this handbook page for in-depth instructions. (Pro tip: read the entire section first, then start following the directions). As the section mentions, much of what you need is handled by running the onboarding script.

profiles.yml tips

- Looking for your profiles.yml file? Go to your home directory and hit command + shift + .. This will expose your hidden files (ones that have a period as the first character) and reveal the file you need to update.
- user: This is your GitLab email address (ex: JSMITH@GITLAB.COM)
- role: This is your Snowflake role, usually first initial + last name (ex: JSMITH)
- database: This is your first initial + last name (ex: JSMITH)
- Set up targets for different sized warehouses (ex: one for DEVXS and one for DEVL)
 - Note: You should always default to using an XS warehouse. The example provided in the dbt guide defaults to an XS warehouse.

Setting up development databases in Snowflake

To do testing locally, you have your own PROD and PREP databases available for development. Because of the large data volumes (especially the product models), we strongly recommend that you clone any required dependencies for the model(s) that you are building. DO NOT build these using dbt run, you should only do that for models that you have changed. If you build these using dbt run, it will take a really long time and consume a lot of resources because it completely builds the models from scratch.

Here are a few options for cloning dependencies in Snowflake. In these examples, pretend that your role is JSMITH and your development databases are JSMITHPROD and JSMITHPREP.

Clone a lineage using the command line

Please see instructions [here](#).

on how to clone a single model or an entire lineage (similar to the CI jobs) using the command line. This is the preferred method for cloning models locally.

Note: You need to run this in the /analytics directory. You cannot run it in the dbt virtual environment.

Clone a single model using Snowflake

This will clone a single model from the production database into your development database. Cloning a single model only takes a few seconds.

```
sql
CREATE OR REPLACE TRANSIENT TABLE {DEVDATABASE}.{SCHEMANAME}.{TABLENAME}
CLONE {PRODDATABASE}.{SCHEMANAME}.{TABLENAME}
;
```

--Example

```
CREATE OR REPLACE TRANSIENT TABLE JSMITHPROD.common.dimpingmetric
CLONE PROD.common.dimpingmetric
;
```

Note: the schema must already exist in your development database in order to clone the model

<details markdown="1"><summary>How to create a new schema in your development database</summary>

```
sql
CREATE SCHEMA IF NOT EXISTS {DEVDATABASE}.{SCHEMANAME};
```

--Example

```
CREATE SCHEMA IF NOT EXISTS JSMITHPROD.COMMON;
```

</details>

Clone an entire schema using Snowflake

This will clone all models in a production schema into a schema in your development database. Cloning an entire schema can take several minutes.

```
sql
CREATE OR REPLACE SCHEMA {DEVDATABASE}.{SCHEMANAME}
CLONE {PRODDATABASE}.{SCHEMANAME}
```

```
;
```

```
--Example
```

```
CREATE OR REPLACE SCHEMA JSMITHPROD.COMMON
```

```
CLONE PROD.COMMON
```

```
;
```

Running and testing models in dbt

You should always default to using an XS warehouse and be mindful of the costs associated with running queries using a larger warehouse. See the [dbt Guide](#)

for guidance on selecting the appropriate warehouse when running dbt.

Running models in dbt

Once you make the desired changes to the model(s) you are working on, you need to build the model(s) locally to make sure the build succeeds and do local testing. To do this, you can use the dbt run command. Here are some other useful resources and tips:

- [GitLab Data Team dbt command line cheat sheet](#)
- [Model selection syntax overview](#)
- You can specify multiple models by including them in a command with a space in between the names
- You can specify multiple models by using graph operators like + and @ to refer to upstream or downstream models
- You can specify that you want to run the models using a larger warehouse by setting the target (warehouse connection).

console

dbt run --select mymodel	run mymodel
dbt run --select mymodel mymodel2	run mymodel and mymodel2
dbt run --select mymodel+	run mymodel and all children
dbt run --select +mymodel	run mymodel and all parents
dbt run --select +mymodel+ children	run mymodel, all of its parents, and all of its children
dbt run --select @mymodel	run mymodel, all parents, all children, AND all parents of all children
dbt run --select mymodel+ --exclude mymodel2	run mymodel and all children EXCEPT mymodel2
dbt run --select mymodel --target dev	run mymodel using a L warehouse (targets defined in profiles.yml)

Testing models in dbt

Testing models in dbt just requires you to run dbt test. dbt test uses the same syntax (ex: --select) as dbt run

console

```
dbt test --select mymodel run custom tests on mymodel
```

Linting

Once you are running dbt, linting a model can be done with a single command. Please read the SQLFluff section of the SQL Style Guide

for instructions on how to install SQLFluff on your local machine and more details about the linter settings. When you run the linter, the results will be printed in your terminal.

You can also leverage the fix command to have the linter apply fixes to rule violations (when possible).

console

```
sqlfluff lint models/path/to/file/file-to-lint.sql lint the file and print results in terminal
```

```
sqlfluff fix models/path/to/file/file-to-lint.sql lint the file and apply fixes
```

Real-world walkthrough

Here is the agenda from a dbt working session

with the Data team and functional analysts. You can view the recording [here](#), the live demo of updating and testing a model starts at ~30:00. (Agenda and recording are internal only).

Updating dbt documentation

There is one main file used to generate dbt Docs: schema.yml (ex:

transform/snowflake-dbt/models/commonmart/schema.yml).

You can read the dbt docs on dbt Docs [here](#)

TL; DR: Our dbt docs work by setting the model description in schemaname.md, setting the column definitions in commoncolumns.md, then referencing those files in schema.yml. Additional details and examples below.

schema.yml

schema.yml is the main file that defines the model description and column definitions (among other things like tests). You can add the description and column definitions directly to this file OR you can use the doc function to reference documentation in a different file. Example schema.yml file [here](#).

yaml

models:

- name: marteventvalid

- description: '{{ doc("marteventvalid") }}'

- columns:

- name: eventpk

```
description: '{{ doc("eventpk") }}'
tests:
  - notnull
  - unique
- name: dimeventdateid
  description: '{{ doc("dimeventdateid") }}'
```

Given the level of detail in our documentation, the schema.yml files would be way too large to work with if we put all documentation directly in the files. To mitigate that potential issue, the GitLab central Data team uses two additional files:

1. schemaname.md: transform/snowflake-dbt/models/path/to/modeldirectory/schemaname.md.

We use these markdown files for model descriptions. (Ex:

transform/snowflake-dbt/models/commonmart/commonmart.md)

- There are a few exceptions where the file name does not exactly match the schema name. You should look for the markdown file in the model schema's directory

1. commoncolumns.md: transform/snowflake-dbt/models/common/commoncolumns.md.

We use this markdown file for column definitions.

schemaname.md

schemaname.md is the file we use to define the model description. We can then reference the documentation in schema.yml. Example schemaname.md file [here](#).

To add a new model In the markdown file, add the following:

markdown

```
{% docs modelName %}
```

Model description here

```
{% enddocs %}
```

Then you can reference a definition in this schema.yml using '{{ doc("modelName") }}' (example below).

yaml

models:

```
- name: martevalid
  description: '{{ doc("martevalid") }}'
  columns:
```

commoncolumns.md

commoncolumns.md

is the file we use to define the column definitions. It works the same way as schemaname.md in

that we can reference the definitions in schema.yml. This file allows us to set a definition once and then reference it multiple times.

Before adding a new column definition, please check to see if a definition already exists. If it is not, then add the following to the markdown file:

```
markdown
{% docs columnname %}
```

Column definition

```
{% enddocs %}
```

Once the column is defined, you can reference it in schema.yml using '{{ doc("columnname") }}' (example below)

```
yaml
models:
- name: marteventvalid
  description: '{{ doc("marteventvalid") }}'
  columns:
  - name: eventpk
    description: '{{ doc("eventpk") }}'
```

Model configuration

The model configuration can be set at the top of the .sql file. Configurations include whether the model should be a materialized table vs a view, a full refresh vs an incremental load, etc.

Please refer to the dbt Guide for details on model configuration.

Tags for product data models

With respect to product data models, you need to include the "product" tag on the model to ensure that it runs in the correct DAG (i.e., it builds after the other product dependencies).

Failure to include this tag could lead to a lag in data freshness (ex: if the model builds before a dependency is refreshed). Please see existing models in dbt docs for additional examples.

```
sql
{{ config(
  tags=["product"]
) }}
```

There is additional documentation on tags on the dbt Guide handbook page.

MR workflow

The Data team has a well-documented MR workflow.

MR templates

Make sure you use the appropriate MR template based on your use case. The templates also have detailed instructions on testing, pipelines, etc. For that vast majority of use cases, you will use one of the following templates:

- dbt Model Changes:

Use this for changes to common or legacy (non-workspace) models

- dbt Workspace Changes:

Use this for changes to workspace schemas (ex: workspaceproduct)

- All Other Changes:

Use this for updates to dbt docs that do not actually change the models

MR pipelines & CI jobs

Please see the Data Team CI Jobs handbook page for the most up-to-date information and details about the different CI jobs.

```
{{< panel header="Note" header-bg="blue" >}}
```

You need to wait for a job to finish before running the next one. For example, you need to wait until the dependencies are all cloned before you can build the changed model.

```
{{< /panel >}}
```

MRs to add new tables or columns to the Postgres/GitLab.com pipeline

In order to facilitate self-service on requests to add a new table or column to the Postgres/GitLab.com pipeline, the Data team created a runbook and videos to outline the required steps:

- Runbook
- Video (Part 1)
- Video (Part 2)

Getting a review

Before assigning any reviewers, make sure that you go through the checklist in the MR templates.

This will ensure that you completed all required steps before a member of the Data team starts reviewing the code.

In the Auditing section of the MR template, make sure that all queries refer to the MR branch database.

Other team members (ex: Analytics Engineers) reviewing the MR will also have access to the MR database and that is how they will review and validate the changes.

Once the MR is ready for review, tag code owners and assign as reviewers (you will see

code owners listed in the approval section of the MR). For changes to workspace models, please be sure to have a functional analyst review the MR before tagging a member of the Data team as a reviewer.

Getting the MR merged

Once the MR is approved by the required number of code owners (as specified in the approval section of the MR), you can assign to a maintainer for final review and merge.

SECTION: product/groups/product-analysis/experimentation.md

Overview

At GitLab, we have a unique approach to experimentation that is built in-house by our incredible development team. The reason we use this approach is to uphold our commitment to our users and customers to protect their privacy. This custom approach leads to some challenges that are not experienced with more commonly used 3rd party experimentation tools. Due to this reality, experimentation at GitLab must be approached with a high level of intentionality and forethought. The purpose of this handbook page is to create some guidelines around experimentation to avoid common errors and to define best practices.

In order to increase our velocity while maintaining our ability to learn from experiments, the GitLab Growth stage (including the Product Analysis group) is adopting a new framework for designing and analyzing experiments. This framework is adapted from the work of data scientist Danielle Nelson.

Like everything at GitLab, this page and how we implement the framework are in draft, and we will iterate over time.

Framework

This experiment framework is derived from the previous version of our Experiment Framework, which utilized the `gitlabdotcomexperimentsubjects` table to host the unique identifiers. The `snowplowgitlabeventsexperimentcontextsall` replaces this table as a landing place for all unique identifiers. This framework was created to improve efficiency and maintain our commitment to user privacy by using pseudonymized data to avoid tracking user-identifiable data.

The current experiment framework that is being utilized by GitLab is called the `gitlab-experiment-gem` or GLEX for short. Here at GitLab we run experiments as A/B/n tests and review the data the experiment generates. From that data, we determine the best performing code path and promote it as the new default code path, or revert back to the original code path. You can read our Experiment Guide documentation if you're curious about how we use this gem internally at GitLab. This experiment framework relies heavily on front-end events or events that are created by our data collector, Snowplow.

When we discuss the behavior of this gem, we'll use terms like experiment, context, control, candidate, and variant. It's worth defining these terms so they're more understood. These are

the universal terms used across the company.

- experiment is any deviation of code paths we want to run sometimes and not others.
- contexts is used to identify a consistent experience we'll provide in an experiment.
- control is the default, or "original" code path.
- candidate defines that there's one experimental code path.
- variant(s) is used when more than one experimental code path exists.
- behaviors is used to reference all possible code paths of an experiment.

For this documentation, these are the important terms that need to be expounded on that are specific to Snowplow Event Structures.

Contexts

This is the JSON column that will contain any useful identifiers that are used to tie experiments to a specific namespace or project. These identifiers will be used to measure experiment impact on the KPIs and is required for every experiment for analysis. Prior versions of experiment analysis required this data to be parsed manually via SQL in order to utilize the information stored in this column, but the current `experimentcontextsall` table already parses this data for ease of use.

Contextkeys are used in experiment analysis to determine the unique users/projects/namespaces that engage in the various features and/or stages as defined by the experiment analysis. Contextkeys can be "sticky" to various identifiers (either user, namespace, project or some combination of user per project or user per namespace) depending on what is needed for the experiment.

Examples:

- Assigning contextkey to be sticky to a user (1 context key = 1 user), if we're identifying how many users adopted specific features as a result of an experiment regardless of their namespace or project.
- Assigning contextkey to be sticky to namespaces (1 context key = 1 namespace) , if we're identifying how many stages were adopted as a result of an experiment regardless of how many users engaged in each stage.
- Assig